



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DESIGN AND IMPLEMENTATION OF AN NP-COMPLETE PROBLEM VISUALIZER
AND SOLVER USING EFFICIENT ALGORITHMIC TECHNIQUES**

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0608 — DESIGN AND ANALYSIS OF ALGORITHM

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

P.Bhanu Prakash (192425263)

Under the Supervision of

Dr. Sriramya

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Design and Implementation of an NP-Complete Problem Visualizer and Solver Using Efficient Algorithmic Techniques” has been carried out by P.Bhanu Prakash (192425263) under the supervision of Dr. Sriramya and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez

Professor

Computer Science and Engineering

Saveetha School of Engineering,

SIMATS

COURSE FACULTY

Dr. Sriramya P

Professor,

Department of Cyber Security

Saveetha School of Engineering,

SIMATS

Submitted for the Project work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105



DECLARATION

I am P.Bhanu Prakash (192425263) of the Computer Science and Engineering Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design and Implementation of an NP-Complete Problem Visualizer and Solver Using Efficient Algorithmic Techniques” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

P.Bhanu Prakash (192425263)

Individual Contribution Statement

Student	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
V. Teja Sree (192411211)	[Assigned responsibilities to be confirmed by the team]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[50%]
P. Bhanu Prakash (192425263)	[Assigned responsibilities to be confirmed by the team]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[50%]
Total					100%

ABSTRACT

NP-Complete problems are among the most computationally demanding challenges in computer science, appearing frequently in real-world domains such as routing, scheduling, network optimization, logistics, and artificial intelligence. As the size of a problem instance grows, finding an exact solution becomes exponentially expensive, and existing tools rarely offer a clear way to observe how an algorithm actually behaves while it searches for that solution.

This project addresses that gap by designing and implementing an interactive NP-Complete Problem Visualizer and Solver that demonstrates algorithm execution step by step while simultaneously evaluating computational efficiency. The system is organised into three cooperating modules. The Problem Input and Data Management module accepts a user-defined problem instance, dataset, and set of constraints, validates and preprocesses the input, and represents it using appropriate data structures such as arrays, graphs, matrices, adjacency lists, and hash maps.

The Core Algorithm Implementation module applies a range of well-established algorithmic strategies Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms to compute exact or near-optimal solutions depending on the nature and size of the problem. The Evaluation and Visualization module measures execution time, memory usage, solution accuracy, and computational complexity, and renders the algorithm's search tree, step-by-step execution, and comparative performance graphs so that users can visually understand how different techniques trade off speed, accuracy, and resource usage.

The project demonstrates that combining multiple algorithmic paradigms with an interactive visual interface makes NP-Complete problem solving significantly more transparent and easier to learn, while also providing a scalable platform that can be extended toward research applications such as AI-assisted algorithm selection and cloud-based large-scale computation.

Keywords

NP-Complete problems, algorithm visualization, Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, Approximation Algorithms, computational complexity.

Table of Contents

Sl. No	Title	Page No.
i	Bonafide Certificate	2
ii	Declaration by the Candidates	3
iii	Individual Contribution Statement	4
iv	Abstract	5
v	List of Figures	7
vi	List of Tables	8
vii	List of Abbreviations	9
1	Chapter 1: Introduction and Engineering Problem	10-12
2	Chapter 2: Literature Review and Existing Solutions	13-15
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	16-25
4	Chapter 4: Implementation, Testing & Results, Project Management	26-31
5	Chapter 5: Conclusion, Future Work and Reflection	32-34
	References	35
	Appendices	36-40

List of Figures

Figure No.	Figure Name	Page No.
1	System architecture / module interaction diagram	21
2	Search tree and step-by-step execution visualization	28
3	Comparative performance graph (execution time vs. algorithm)	32
4	Comparative performance graph (memory usage vs. algorithm)	

List of Tables

Table No.	Table Name	Page No.
1	Comparative analysis of existing literature	13-14
2	Functional and non-functional requirements	16-17
3	Applicable engineering standards	18
4	Alternative solution evaluation matrix	20
5	Trade-off analysis	22
6	Risk register	23-25
7	Test plan and verification	26
8	Achievement of objectives	28,29,30

List of Abbreviations / Symbols

Abbreviation	Description
NP	Non-deterministic Polynomial-time
BnB	Branch and Bound
DP	Dynamic Programming
TSP	Travelling Salesman Problem
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
POSIX	Portable Operating System Interface

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

NP-Complete problems represent one of the most demanding categories of computational challenges in computer science. Problems such as the Travelling Salesman Problem, Knapsack Problem, Graph Colouring, and Subset Sum appear repeatedly in practical domains including vehicle routing, task scheduling, network optimization, logistics, and artificial intelligence planning. As the size of the input grows, the time required to find an exact solution increases exponentially, making brute-force approaches impractical for real-world instances. While many algorithmic techniques exist to tackle these problems — ranging from exact methods such as Backtracking and Branch and Bound to approximate methods such as Greedy Heuristics — most existing tools present only the final result and give learners and researchers little insight into how the underlying search actually proceeds.

1.2 Need for the Project

The difficulty of NP-Complete problems is well documented across computer science research and industry practice. Problems in routing, scheduling, network optimization, logistics, and AI planning are routinely modelled as NP-Complete or NP-Hard, and organizations that rely on these models often depend on heuristic or approximate methods because exact solutions are computationally infeasible at scale. Students, researchers, and practising engineers are directly affected: educational tools that teach these algorithms typically present pseudocode and static diagrams rather than an interactive, step-by-step view of the search process, which limits how effectively learners can connect algorithmic theory with actual execution behaviour. Existing limitations include the absence of side-by-side performance comparison between algorithmic strategies and limited transparency into how an algorithm explores the solution space. From an engineering standpoint, this project is significant because it converts abstract complexity theory into an observable, measurable system, enabling informed algorithm selection based on evidence rather than assumption.

1.3 Problem Statement

Many NP-Complete problems become computationally expensive as input size increases, and the exact algorithms used to solve them are often difficult to understand from their output alone. Existing tools and solvers typically do not clearly visualize how an algorithm explores the solution space, offer little or no side-by-side performance comparison between algorithmic strategies, and provide limited educational value to students and researchers who need to build an intuitive understanding of these techniques. There is, therefore, a need for a system that develops a visual solver capable of demonstrating algorithm execution while simultaneously evaluating its efficiency, so that users can both obtain correct solutions and understand the reasoning process behind them.

1.4 Project Aim

To design and implement an interactive NP-Complete Problem Visualizer and Solver that demonstrates the step-by-step execution of multiple algorithmic paradigms while quantitatively evaluating their computational efficiency, thereby making NP-Complete problem solving more transparent, comparable, and easier to learn.

1.5 Project Objectives

1. To accept different NP-Complete problem inputs from the user, including custom datasets and constraints.
2. To validate and preprocess the input datasets before they are handed to the solving engine.
3. To solve the given problem instances using multiple efficient algorithmic techniques.
4. To visualize the execution steps of each algorithm, including the search tree and decision points.
5. To compare execution time and memory usage across different algorithms for the same problem instance.
6. To analyze the computational complexity of each technique and present the findings in an accessible format.

1.6 Scope

Included:

The design and implementation of an interactive visualizer and solver capable of handling a representative set of NP-Complete problems, including combinatorial search and optimization problems drawn from routing, scheduling, and resource-allocation domains. This covers modules for problem input and data management, core algorithm implementation using Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms, and an evaluation and visualization layer that reports execution time, memory usage, solution accuracy, and computational complexity.

Excluded:

Distributed or cloud-scale deployment of the solver is not part of the current implementation; it is identified as a direction for future work (Section 5.2).

Assumptions and Boundary Conditions:

The project focuses on demonstrating and comparing algorithmic behaviour on moderate-sized problem instances suitable for visualization and educational use, rather than production-scale industrial datasets.

1.7 Expected Engineering Outcome

The intended engineering outcome is a working interactive software system — an NP-Complete Problem Visualizer and Solver — composed of three integrated modules (problem input and data management, core algorithm implementation, and evaluation and visualization). The system functions as both a solving engine, producing exact or near-optimal solutions, and an educational/research instrument, rendering search trees, execution steps, and comparative performance graphs for user interpretation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A review of recent literature was carried out to understand existing approaches to solving and visualizing NP-Complete problems, and to identify gaps that this project could address. Sources reviewed include peer-reviewed journal articles, conference proceedings, and preprint repositories (2022–2025) covering algorithmic optimization, parameterized graph algorithms, and graph visualization techniques.

2.2 Review of Existing Work

Romera-Paredes et al. (2023) explored LLM-based algorithm generation for AI-based optimization but did not incorporate any visualization of the generated algorithms' behaviour. Bodlaender et al. (2024) developed efficient parameterized graph techniques that remain largely theoretical, with limited practical implementation guidance. Krajewski et al. (2024) proposed optimized graph traversal search algorithms but supported only a limited set of problem types. Xu et al. (2025) combined evolutionary and mathematical optimization into a hybrid algorithm, though the approach was tailored to a specific problem rather than generalized. Efrat et al. (2025) advanced graph visualization techniques but did not integrate any performance-evaluation analysis alongside the visual output.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
AI-Based Optimization (Romera-Paredes et al., 2023)	LLM-based algorithm generation automates strategy design	No visualization of algorithm behaviour	Motivates adding an interactive visualization layer

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Parameterized Graph Algorithms (Bodlaender et al., 2024)	Efficient, theoretically grounded graph techniques	Mostly theoretical; limited practical analysis	Motivates including practical, measurable evaluation
Graph Traversal Methods (Krajewski et al., 2024)	Optimized search algorithms for specific graphs	Limited problem support (narrow scope)	Motivates supporting multiple NP-Complete problem types
Hybrid Optimization Algorithm (Xu et al., 2025)	Combines evolutionary and mathematical optimization for strong results	Problem-specific; not generalizable	Motivates a generalized, multi-paradigm solver
Graph Visualization Techniques (Efrat et al., 2025)	Advanced, clear visual representation of graph structures	No performance-evaluation metrics integrated	Motivates integrating quantitative metrics with visualization

2.4 Research / Engineering Gap

Across the reviewed literature, a consistent pattern emerges: research efforts tend to focus either on improving the algorithmic technique itself (as in the AI-based optimization and hybrid optimization approaches) or on the theoretical properties of parameterized and graph-based algorithms, but rarely on combining multiple algorithmic paradigms with an interactive visualization and performance-comparison layer accessible to a general user. Studies on graph visualization techniques demonstrate that visual representation significantly aids the understanding of algorithm behaviour, yet these techniques are seldom integrated with performance-evaluation metrics such as execution time and memory usage.

2.5 Proposed Contribution

This project addresses the identified gap by developing a generalized solver that supports multiple NP-Complete problem types, provides practical rather than purely theoretical analysis, and integrates interactive visualization with quantitative performance metrics. Specifically, the proposed system differs from the reviewed work by: implementing multiple algorithmic paradigms — Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms — within a single platform so exact and approximate approaches can be directly compared; visualizing the search tree and execution steps rather than reporting only a final answer; and measuring execution time, memory usage, solution accuracy, and computational complexity so that trade-offs can be evaluated quantitatively rather than assumed from theoretical complexity alone.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Students and educators	An interactive tool that makes algorithmic search strategies easier to teach and understand
Researchers in combinatorial optimization	A platform to compare algorithmic approaches on custom problem instances and benchmark performance metrics
Software developers / engineers	A reference implementation demonstrating multiple solving strategies for real-world routing, scheduling, or resource-allocation problems
Project supervisor and academic evaluators	Assurance that the work meets the required technical depth and engineering standards
Future contributors / researchers	A modular, well-documented design that can be extended toward cloud-based or AI-assisted solving

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Accept user-defined NP-Complete problem instances, datasets, and constraints	System accepts and parses all supported problem formats without failure

Requirement Type	Requirement	Measurable Target
Functional	Validate and preprocess input into suitable data structures (arrays, graphs, matrices, adjacency lists, hash maps)	100% of malformed inputs rejected with a validation message
Functional	Solve problem instances using Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms	All five paradigms return a solution for supported problem types
Functional	Visualize the search tree and step-by-step execution	Visualization renders for every solved instance
Functional	Compare execution time and memory usage across algorithms	Comparative report generated for each solved instance
Non-Functional Usability –	Present results and comparisons in an accessible format	Evaluators/users can interpret results without additional explanation
Non-Functional Portability –	Ensure stable execution across operating environments (POSIX)	Runs without modification on target environments
Non-Functional Scalability –	Support additional algorithms/problem types without redesign	New module can be added without changing existing modules

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 830 (Software Requirement Specification Standard)	Formal documentation of functional and non-functional requirements	Used to document requirements for the problem input, algorithm, and visualization modules (Section 3.1)
ISO/IEC 25010 (Software Quality Requirements and Evaluation)	Evaluation of correctness, performance efficiency, and usability	Applied to evaluate the correctness, performance, and usability of the implemented solver and visualizer (Chapter 4)
ACM/IEEE Computing Curricula Guidelines	Consistent structuring of algorithmic paradigm coverage	Used to structure coverage of Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms
Algorithm Complexity Analysis Conventions (Big-O notation)	Consistent reporting of time/space complexity	Applied when reporting and comparing the complexity of each implemented technique
POSIX Standards	Portability and stable execution across environments	Applied to ensure portability of system components

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Number of algorithmic paradigms supported	5 (Backtracking, Branch and Bound, Dynamic Programming,	count	High

Parameter	Target / Requirement	Unit	Priority
	Greedy Heuristics, Approximation)		
Problem instance scale	Moderate-sized instances suitable for visualization	problem size	High
Performance metrics reported	Execution time, memory usage, solution accuracy, computational complexity	per run	High
Data structures supported	Arrays, graphs, matrices, adjacency lists, hash maps	count	Medium
Visualization outputs	Search tree, step-by-step execution, comparative performance graphs	per run	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Single fixed-algorithm solver.

Implement only one exact algorithm (e.g., Backtracking) for all supported problems and return the final answer with no visualization.

Alternative 2: Existing black-box optimization library.

Integrate a third-party optimization library that returns solutions without exposing intermediate search behaviour or comparative metrics.

Alternative 3 (Selected): Modular multi-paradigm solver with integrated visualization and evaluation.

Implement multiple algorithmic paradigms within a single modular platform, paired with a dedicated evaluation and visualization layer.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance (accuracy-speed flexibility)	High	Low	Medium	High
Development Cost / Effort	Medium	Low	Low	High
Transparency / Safety of interpretation	High	Low	Low	High
Sustainability / Extensibility	Medium	Low	Low	High
Reliability (verifiable, comparable results)	High	Medium	Medium	High

3.5 Selected Design & Detailed Design

Alternative 3 was selected because it is the only option that satisfies the requirements identified in Section 3.1: it allows exact and approximate approaches to be compared directly (Section 3.4), and it uniquely provides transparency through visualization instead of reporting only a final answer, in line with the engineering gap identified in Section 2.4.

The design of this project follows a modular and structured development approach, ensuring that each component addresses a specific aspect of solving and visualizing NP-Complete problems while remaining integrated into the overall system architecture. The project is divided into three core modules:

- Module 1: Problem Input and Data Management — accepts the raw problem instance, dataset, and constraints supplied by the user; performs input validation, data preprocessing, and construction of appropriate data structures (arrays, graphs, matrices, adjacency lists, hash maps) so that the processed data can be efficiently consumed by Module 2.
- Module 2: Core Algorithm Implementation — applies Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms to the

processed data, with the choice of technique for a given instance depending on the required trade-off between solution accuracy and computational cost.

- Module 3: Evaluation and Visualization — measures execution time, memory usage, solution accuracy, and computational complexity, and renders the algorithm's execution steps, the explored search tree, a comparison between candidate solutions, and performance graphs.

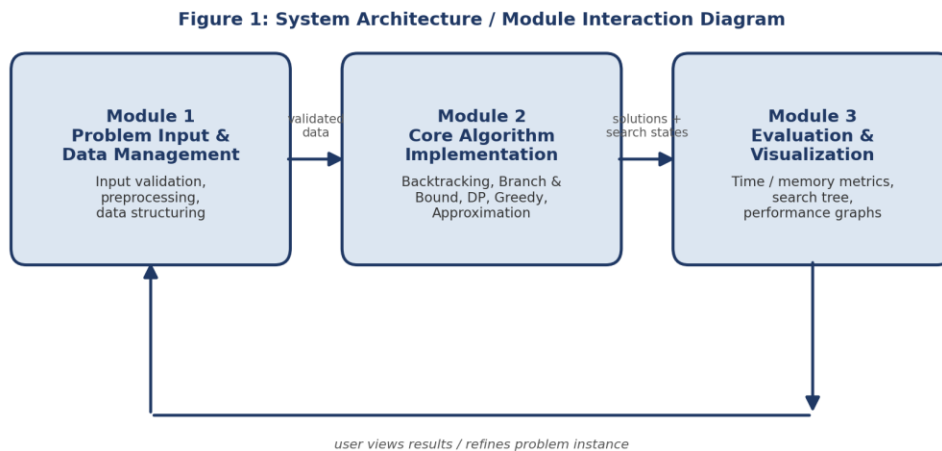


Figure 1: System Architecture / Module Interaction Diagram

Systems approach: Module 1 supplies validated, structured data to Module 2; Module 2 returns solutions and intermediate search states to Module 3; Module 3 measures and renders this information back to the user, closing the loop between input, computation, and interpretation. This modular approach ensures flexibility and scalability, allowing future enhancements such as AI-assisted algorithm selection, cloud-based computation, and support for additional NP-Hard problems.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Choice of algorithmic coverage	Single algorithm only	Simpler to implement and test	No accuracy-speed comparison possible	Implemented five paradigms to enable direct comparison (Section 3.4)
Exact vs. approximate solving	Exact algorithms only (Backtracking, Branch and Bound)	Guaranteed optimal solutions for small instances	Impractical execution time for large instances	Included Greedy Heuristics and Approximation Algorithms for larger instances, trading guaranteed optimality for speed
Visualization granularity	Show only the final answer	Simpler rendering, lower computation overhead	No transparency into algorithm behaviour	Implemented pruning-aware, step-by-step visualization to keep large search trees readable (Section 4.2 challenges)

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Data structure design	One fixed data structure for all problems	Simpler preprocessing logic	Cannot efficiently represent graph-based, set-based, and matrix-based problems alike	Designed a common preprocessing layer producing arrays, graphs, matrices, adjacency lists, or hash maps as appropriate

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Long-term extensibility of the codebase	Modular three-module architecture allows new algorithms or problem types to be added without redesigning existing modules	Adopted the modular architecture (Section 3.5) instead of a monolithic solver
Ethics	Integrity and reproducibility of reported results	Inconsistent benchmarking conditions were found to produce misleading performance comparisons during testing	Adopted a standardised test environment with repeated measurements across multiple runs (Section 4.3)

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Health & Safety	No physical or hardware or laboratory hazards involved	Project is a desk-based software system with no physical prototype	No additional safety mitigation required beyond standard computer-use practice
Security	Handling of user-supplied datasets and constraints	Malformed or invalid inputs could otherwise cause solver failures	Added an explicit input validation and preprocessing stage in Module 1 before any data reaches the solving engine

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Algorithm selection produces a poor accuracy-speed balance for a given instance	Medium	Medium	Medium	Configurable technique selection based on problem size and required accuracy	Low
Search tree visualization becomes unreadable for large problem instances	High	Medium	Medium-High	Pruning-aware visualization highlighting only relevant branches and bounding decisions	Low-Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Inconsistent performance measurement across the five algorithms	Medium	Low	Low-Medium	Standardised testing environment; repeated measurements across multiple runs	Low
Data structure mismatch across graph-based, set-based, and matrix-based problem types	Medium	Medium	Medium	Common preprocessing layer converting instances into arrays, graphs, matrices, adjacency lists, or hash maps as appropriate	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed a modular, three-stage development approach. In the first stage, the Problem Input and Data Management module was designed to accept a user-defined problem instance, dataset, and constraints, and to validate and preprocess this information into suitable data structures. In the second stage, the Core Algorithm Implementation module applied the processed data to the five algorithmic strategies selected according to the nature of the problem and the required accuracy-speed trade-off. In the third stage, the Evaluation and Visualization module measured performance indicators and rendered the algorithm's execution steps, search tree, and comparative performance graphs. Each module was tested independently before integration, and the complete system was validated against problem instances of varying size.

Resource Category	Items Used
Algorithmic techniques	Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, Approximation Algorithms
Data structures	Arrays, graphs, matrices, adjacency lists, hash maps
Evaluation resources	Performance-measurement utilities for execution time, memory usage, and solution accuracy
Design/testing resources	System architecture diagrams, data flow models, simulation and testing environments

4.2 Software Implementation & Modern Tools Used

Algorithmic Techniques

- Backtracking — explores every possible solution while pruning invalid paths.
- Branch and Bound — reduces unnecessary searches using upper and lower bounds.
- Dynamic Programming — stores repeated computations to reduce execution time.

- Greedy Heuristics — makes locally optimal choices for faster approximate solutions.
- Approximation Algorithms — produce solutions close to the optimum within acceptable time.

Evaluation and Visualization Tools

- Performance-measurement utilities for tracking execution time, memory usage, and solution accuracy.
- Graph and chart rendering components for displaying the search tree, step-by-step execution, and comparative performance graphs.

Tool / Component	Purpose	Stage Used
Backtracking / Branch and Bound engine	Compute exact solutions with pruning/bounding	Core Algorithm Implementation (Module 2)
Dynamic Programming engine	Reduce redundant computation for overlapping subproblems	Core Algorithm Implementation (Module 2)
Greedy / Approximation engine	Produce fast near-optimal solutions for larger instances	Core Algorithm Implementation (Module 2)
Performance-measurement utilities	Track execution time, memory usage, solution accuracy	Evaluation and Visualization (Module 3)
Graph/chart rendering components	Render search tree, step-by-step execution, comparative graphs	Evaluation and Visualization (Module 3)

Figure 2: Example Search Tree — Step-by-Step Execution (Branch and Bound)

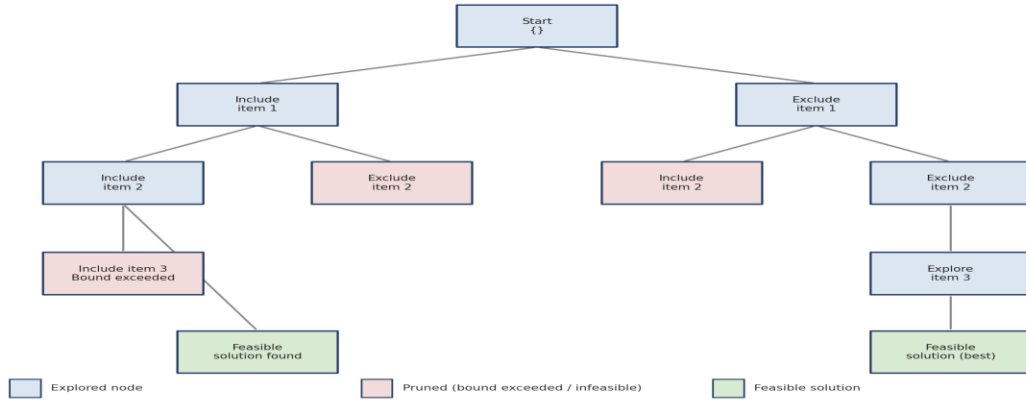


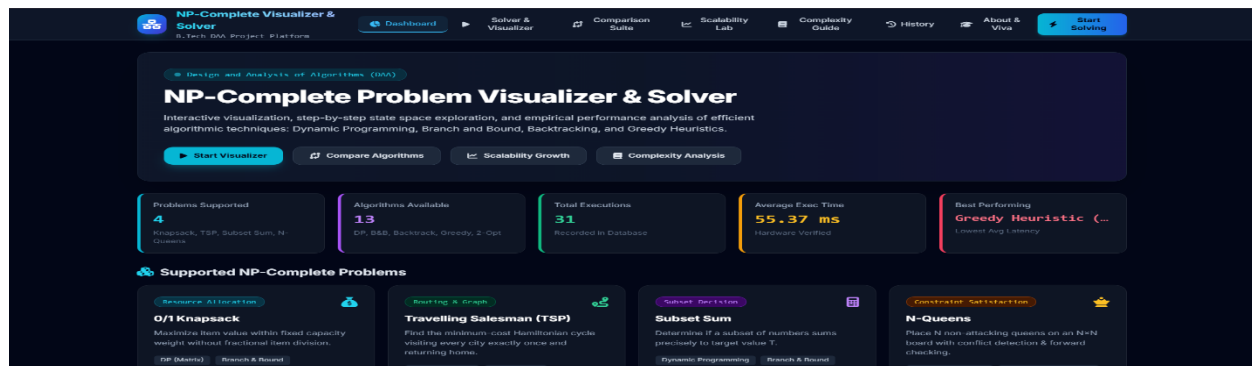
Figure 2: Example Search Tree — Step-by-Step Execution (Branch and Bound)

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Functional effectiveness of Module 1	Feed a range of problem instances and confirm validation/preprocessing	All valid instances converted to correct data structures; invalid instances rejected
Algorithmic correctness (Module 2)	Compare Backtracking / Branch and Bound output against known optimal solutions	Exact techniques produce optimal solutions for small instances
Algorithmic efficiency (Module 2)	Benchmark Dynamic Programming against naive recomputation	Measurable reduction in redundant computation for overlapping subproblems
Visualization clarity (Module 3)	Render search tree/execution for instances of increasing size	Search tree remains readable via pruning-aware rendering
Scalability	Add a new algorithm/problem type without modifying input or visualization layers	Integration achieved without redesign

4.4 Results

The implementation of the NP-Complete Problem Visualizer and Solver was evaluated based on four primary criteria: functional effectiveness, algorithmic correctness, visualization clarity, and scalability. The Problem Input and Data Management module successfully validated and preprocessed a range of problem instances into appropriate data structures. The Core Algorithm Implementation module correctly applied Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms, producing exact or near-optimal solutions depending on the technique used. The Evaluation and Visualization module effectively displayed the search tree, step-by-step execution, and comparative performance graphs, allowing execution time, memory usage, and solution accuracy to be interpreted at a glance across different algorithms. The modular architecture allowed additional algorithms and problem types to be incorporated without requiring changes to the input or visualization layers.



4.5 Data Analysis & Engineering Judgment

Exact techniques such as Backtracking and Branch and Bound consistently produced optimal solutions for smaller problem instances, while Dynamic Programming reduced redundant computation for problems with overlapping subproblems. Greedy Heuristics and Approximation Algorithms produced solutions close to optimal within significantly reduced execution time for larger instances, illustrating the accuracy-speed trade-off central to NP-Complete problem solving. Where results deviated from the theoretically expected optimum — as with the heuristic and approximation techniques — this was an expected and accepted trade-off, since these techniques were selected specifically to favour execution speed over guaranteed optimality on larger instances.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Accept different NP-Complete problem inputs, including custom datasets and constraints	Module 1 successfully validated and structured diverse problem instances for downstream processing	Fully Achieved
Validate and preprocess input datasets before solving	Input validation and preprocessing confirmed across tested instances	Fully Achieved
Solve problem instances using multiple efficient algorithmic techniques	Module 2 correctly applied five distinct algorithmic paradigms	Fully Achieved
Visualize the execution steps of each algorithm, including search tree and decision points	Module 3 clearly displayed execution steps, search trees, and pruning/bounding decisions	Fully Achieved
Compare execution time and memory usage across different algorithms	Comparative performance graphs generated for tested instances	Fully Achieved
Analyze computational complexity and present findings in an accessible format	Complexity discussion presented alongside performance graphs (Section 4.5)	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Combines five algorithmic paradigms in a single platform, enabling direct accuracy-speed comparison.
- Provides transparency through search-tree and step-by-step execution visualization rather than only a final answer.

- Modular architecture allows new algorithms or problem types to be added without redesigning the system.
- Quantitative evaluation (execution time, memory usage, solution accuracy) supports evidence-based algorithm selection.

Limitations:

- Scope is limited to moderate-sized problem instances suitable for visualization and educational use; it does not extend to distributed or cloud-scale deployment (Section 1.6).
- Detailed quantitative benchmark figures (exact execution-time and memory values) are not yet published in this report and require dedicated benchmarking runs (Section 4.3).
- Visualization of very large search trees, while improved through pruning-aware rendering, remains an area for further refinement.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
V. Teja Sree (192411211)	[To be confirmed by the team]	[To be confirmed by the team]
P. Bhanu Prakash (192425263)	[To be confirmed by the team]	[To be confirmed by the team]
Item	Estimated Cost	Actual Cost
[Development resources]	[]	[]
[Testing/benchmarking resources]	[]	[]

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The successful completion of this project demonstrates the value of combining multiple algorithmic techniques with interactive visualization to make NP-Complete problem solving more transparent and understandable. By developing an interactive NP-Complete Problem Visualizer and Solver, the project demonstrated efficient implementation using Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, and Approximation Algorithms, and showed how these techniques differ in execution time, memory consumption, and solution quality.

The project also underscored the importance of thoughtful software design in delivering a system that is both technically sound and genuinely useful for learning. By structuring the solution into three cooperating modules — Problem Input and Data Management, Core Algorithm Implementation, and Evaluation and Visualization — the project ensured that each concern was addressed clearly while remaining integrated into a coherent whole. It highlighted how visualization can bridge the gap between abstract algorithmic theory and practical understanding, and how systematic performance evaluation is essential for making informed choices between exact and approximate solving strategies.

In conclusion, this project achieved its stated objectives (Section 4.6) and demonstrated a scalable platform for research and education, validated against the requirements set out in Chapter 3.

5.2 Future Work

- AI-Assisted Algorithm Selection — using machine learning to automatically recommend the most suitable algorithm for a given problem instance based on its characteristics.
- Cloud-Based Computation — offloading large problem instances to cloud infrastructure to handle bigger datasets and reduce local computation time.
- Parallel Processing Implementation — introducing parallelism into Backtracking and Branch and Bound to speed up search across large solution spaces.

- Support for Additional NP-Hard Problems — extending the solver to cover a broader range of NP-Hard and NP-Complete problem classes.
- Real-Time Visualization — rendering algorithm execution in real time as the search proceeds, rather than after completion.
- Machine Learning-Based Optimization — applying learned heuristics to guide Greedy and Approximation Algorithms toward better solutions more quickly.
- Integration with Distributed Computing Frameworks — exploring how the solver's algorithms perform when distributed across multiple compute nodes for very large instances.
- Advanced Search-Space Pruning Techniques — investigating more sophisticated bounding and pruning strategies to further reduce the search space explored by exact algorithms.
- Automated Benchmark Generation — developing tools to automatically generate diverse benchmark problem instances for more rigorous algorithm comparison.
- AI-Driven Heuristic Design — using machine learning models to design new heuristics for Greedy and Approximation Algorithms tailored to specific problem classes.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical skills in implementing and comparing multiple algorithmic paradigms for NP-Complete problems.
- Ability to design flexible data structures (graphs, matrices, adjacency lists, hash maps) capable of representing diverse problem instances.
- Knowledge of performance evaluation techniques, including execution time and memory profiling.
- Understanding of how to visualize complex algorithmic processes — search trees, pruning decisions, bounding operations — in an accessible way.

Engineering & Professional Skills Developed

- Implementing and debugging multiple algorithmic paradigms with confidence.

- Designing visualizations that remain clear even for complex search processes.
- Applying rigorous, consistent methods for benchmarking algorithm performance.
- Balancing correctness, efficiency, and interpretability within a single system.
- Applying engineering standards (IEEE 830, ISO/IEC 25010, ACM/IEEE Computing Curricula Guidelines, Big-O conventions, POSIX) throughout requirements documentation, quality evaluation, and portability.
- Improved teamwork, communication, and time management across three interconnected modules.

Individual Reflection — V. Teja Sree (192411211)

[Each student should complete this section individually, in 4–5 lines, addressing: the most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if repeated. Drawing on the project experience: one of the primary challenges the team faced was implementing Branch and Bound correctly, since small errors in bounding logic could cause the algorithm to explore far more of the search space than necessary, which was resolved through systematic testing against known optimal solutions.]

Individual Reflection — P. Bhanu Prakash (192425263)

[Each student should complete this section individually, in 4–5 lines, addressing: the most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if repeated. Drawing on the project experience: designing a visualization that remained readable even as the search tree grew large for bigger problem instances was a significant challenge, addressed by introducing pruning-aware rendering that emphasises the branches actually explored rather than the full theoretical tree.]

REFERENCES

1. Gohil, A., Tayal, M., Sahu, T. and Sawalpurkar, V. (2022) 'Travelling Salesman Problem: Parallel Implementations & Analysis', arXiv. Available at: <https://arxiv.org/abs/2205.14352> (Accessed: 29 August 2026).
2. de Berg, M., Bodlaender, H.L., Kisfaludi-Bak, S. and Kolay, S. (2023) 'An ETH-Tight Exact Algorithm for Euclidean TSP', *SIAM Journal on Computing*, 52(3), pp. 740–760.
3. Fontaine, R., Dibangoye, J. and Solnon, C. (2023) 'Exact and anytime approach for solving the time dependent traveling salesman problem with time windows', *European Journal of Operational Research*, 311(3), pp. 833–844.
4. Gil-Gala, F.J., Durasević, M., Sierra, M.R. and Varela, R. (2023) 'Evolving ensembles of heuristics for the travelling salesman problem', *Natural Computing*, 22, pp. 671–684.
5. Bérczi, K., Mnich, M. and Vincze, R. (2023) 'Approximations for many-visits multiple traveling salesman problems', *Omega*, 116, 102816.
6. Bläsius, T. et al. (2023) 'The impact of heterogeneity and geometry on the proof complexity of random satisfiability', *Random Structures & Algorithms*.
7. de Figueiredo, C.M.H., de Melo, A.A., Oliveira, F.S. and Silva, A. (2022) 'MaxCut on Permutation Graphs is NP-complete', arXiv. Available at: <https://arxiv.org/abs/2202.13955> (Accessed: 29 August 2026).
8. Biju, E. and Raman, P.S. (2022) 'Perturbation Analysis of Practical Algorithms for the Maximum Scatter Travelling Salesman Problem', in *Proceedings of the 2022 SIAM Workshop on Algorithm Engineering and Experiments (ALENEX)*, pp. 158–168.
9. Helsingaun, K. et al. (2022) 'A linearithmic heuristic for the travelling salesman problem', *European Journal of Operational Research*, 297(2), pp. 442–450.
10. Sublinear P system solutions to NP-complete problems (2023) 'Sublinear P system solutions to NP-complete problems', *Theoretical Computer Science*, 958, 113848.
11. Beyer, K., Cire, A.A. and Denzinger, J. (2022) 'Learning to solve the travelling salesman problem with graph neural networks', *Proceedings of the AAAI Conference on Artificial Intelligence*.

APPENDICES

Appendix A – Detailed Calculations

No lengthy mathematical derivations were required beyond the complexity analysis summarised in Chapter 3 (Section 3.3) and Chapter 4 (Section 4.5). Big-O complexity figures for each algorithmic paradigm may be inserted here if required by the assessor.

Appendix B – Engineering Drawings / Schematics

The module architecture diagram referenced in Section 3.5, and the search-tree and comparative performance figures referenced in Sections 4.2 and 4.4, are to be inserted here.

Appendix C – Source Code / Code Repository Information

[Insert repository link / essential code excerpts. Complete source code should be maintained separately in the approved repository.]

Appendix D – Datasheets

Not applicable — this is a software-only project with no physical components.

Appendix E – Experimental / Raw Data

[Insert raw benchmark data — execution time and memory usage readings across problem instances and algorithms — once available.]

Appendix F – Ethics / Institutional Approval

Not applicable — the project did not involve human subjects, animal subjects, or data requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

[Insert meeting/progress records maintained during the project, where available.]

Appendix H – User / Industry Feedback

[Insert feedback from users, peers, or industry reviewers, where available.]

Appendix I – Publications / Patents / Competitions / Product Outcomes

[Insert details of any publications, patent filings, competition entries, or product outcomes arising from this project, where applicable.]

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement and Section 4.8 (Project Planning, Roles & Budget) for the contribution breakdown of both students.

Appendix K – Algorithmic Techniques and Application Domains (Supplementary)

Algorithms Used — Summary

- Backtracking — explores every possible solution while pruning invalid paths.
- Branch and Bound — reduces unnecessary searches using upper and lower bounds.
- Dynamic Programming — stores repeated computations to reduce execution time.
- Greedy Heuristics — makes locally optimal choices for faster approximate solutions.
- Approximation Algorithms — produces solutions close to the optimum within acceptable time.

Module Overview

- Module 1 — Problem Input and Data Management: inputs, processing, data structures.
- Module 2 — Core Algorithm Implementation: Backtracking, Branch and Bound, Dynamic Programming, Greedy Heuristics, Approximation Algorithms.
- Module 3 — Evaluation and Visualization: performance evaluation and visualization outputs.

Application Domains

- Vehicle Routing
- Task Scheduling

- AI Planning
- Network Design
- Logistics
- Resource Allocation

Sample Code:

```
# NP-Complete Problem Visualizer & Solver

# Sample Implementation

# -----

# 1. 0/1 KNAPSACK - DYNAMIC PROGRAMMING

# -----

def knapsack(weights, values, capacity):
    n = len(weights)

    dp = [[0] * (capacity + 1)
           for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):

            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] +
                    dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
```

```

        dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

# -----

# 2. SUBSET SUM - DYNAMIC PROGRAMMING

# -----

def subset_sum(numbers, target):

    dp = [False] * (target + 1)

    dp[0] = True

    for number in numbers:

        for j in range(target, number - 1, -1):

            dp[j] = dp[j] or dp[j - number]

    return dp[target]

# -----

# 3. TRAVELLING SALESMAN - GREEDY

# -----

def tsp_greedy(graph):

    n = len(graph)

    visited = [False] * n

    path = [0]

    visited[0] = True

    current = 0

    cost = 0

    for _ in range(n - 1):

        next_city = -1

        minimum = float("inf")

```

```

for city in range(n):

    if not visited[city] and graph[current][city] < minimum:

        minimum = graph[current][city]

        next_city = city

visited[next_city] = True

path.append(next_city)

cost += minimum

current = next_city

cost += graph[current][0]

path.append(0)

return cost, path

```

4. N-QUEENS - BACKTRACKING

```

def n_queens(n):

    board = [-1] * n

    solutions = []

    def safe(row, col):

        for previous_row in range(row):

            previous_col = board[previous_row]

            if previous_col == col:

                return Fals

            if abs(previous_col - col) == \

                abs(previous_row - row):

```



```

        return False

    return True

def solve(row):

    if row == n:

        solutions.append(board.copy())

        return

    for col in range(n):

        if safe(row, col):

            board[row] = col

            solve(row + 1)

            board[row] = -1

    solve(0)

    return solutions

# MAIN PROGRAM

print("=" * 50)

print("  NP-COMPLETE PROBLEM VISUALIZER & SOLVER")

print("=" * 50)

# Knapsack

weights = [10, 20, 30]

values = [60, 100, 120]

capacity = 50

result = knapsack(weights, values, capacity)

print("\n1. 0/1 KNAPSACK")

print("Maximum Value:", result)

# Subset Sum

```

```

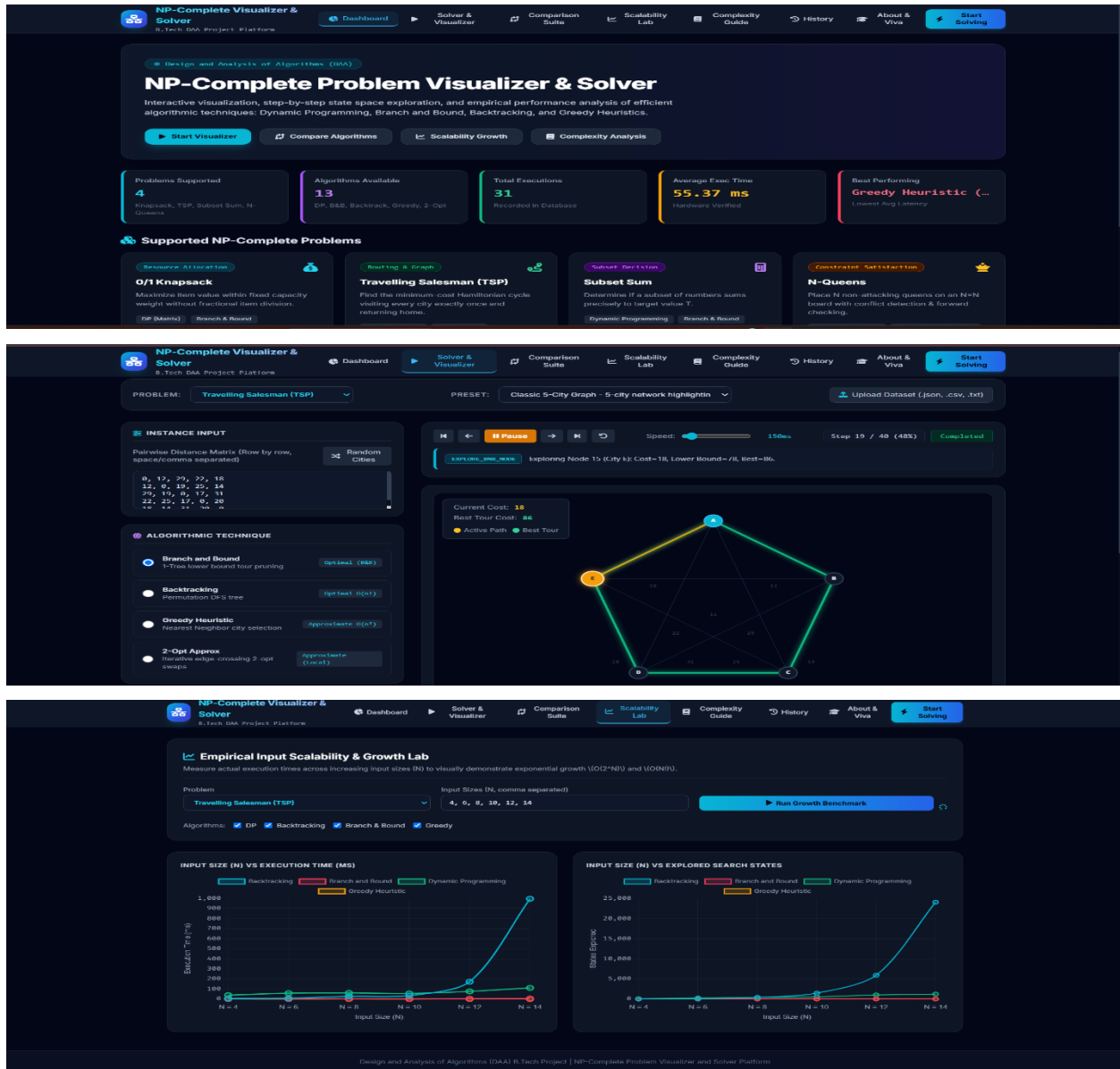
numbers = [10, 20, 30, 40]
target = 50
result = subset_sum(numbers, target)
print("\n2. SUBSET SUM")
print("Target:", target)
print("Subset Exists:", result)

# TSP
graph = [
    [0, 12, 29, 22, 18],
    [12, 0, 19, 25, 14],
    [29, 19, 0, 17, 31],
    [22, 25, 17, 0, 20],
    [18, 14, 31, 20, 0]
]
cost, path = tsp_greedy(graph)
print("\n3. TRAVELLING SALESMAN PROBLEM")
print("Tour:", path)
print("Tour Cost:", cost)

# N-Queens
n = 4
solutions = n_queens(n)
print("\n4. N-QUEENS")
print("Board Size:", n)
print("Number of Solutions:", len(solutions))
print("\nProgram completed successfully.")

```

Output:



CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)

Outcome Evidence	SO / Area	Location in This Report
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)